

ASSIGNMENT

2110413 COMPUTER SECURITY

Assignment 4
Fundamentals of Cryptography

Computer Engineering, Chulalongkorn University

Worralop Srichainont

Student ID: 6632200221

September 14, 2026

Contents

1	Setup	1
2	Part I: Cryptanalysis	1
2.1	Ciphertext	1
2.2	Letter frequency	1
2.3	Short English words	2
2.4	Recovering the plaintext	2
2.5	Caesar brute-force program	3
3	Part II: Kasiski examination	3
4	Part III: Block-cipher modes on a bitmap	4
4.1	Image conversion	4
4.2	Strip the header, encrypt, restore the header	4
4.3	Results	5
4.4	What the pictures show	5
5	Part IV: Hashing, ciphers, and digital signatures	6
5.1	Experimental design	6
5.2	Measured results	6
5.3	Performance versus security	7
5.4	Digital signatures	7

1 Setup

This report answers Activity IV (Fundamentals of Cryptography). Commands were run on macOS (Apple M1 Pro) with Homebrew OpenSSL 3.6.4, ImageMagick 7.1.2-31, Python 3.9.6, and Tectonic 0.17.0. Scripts live in `HW04-Homework-04/code/`. Raw OpenSSL output is in `code/output/speed/`. Generated figures are in `docs/images/`.

The macOS `/usr/bin/openssl` binary is LibreSSL 3.3.6 and does not implement `openssl list`. Every cryptographic command below uses `/opt/homebrew/opt/openssl@3/bin/openssl`.

2 Part I: Cryptanalysis

2.1 Ciphertext

The given ciphertext is a short English message written in a classical cipher:

PRCSOFQX FP QDR AFOPQ CZSPR LA JFPALOQSKR.
QDFP FP ZK LIU BROJZK MOLTROE.

2.2 Letter frequency

Spaces and the period were ignored. There are 59 letters and 20 distinct symbols. Counts came from `code/frequency_analysis.py`.

Rank	Letter	Count
1	P	7
2–4	R, O, F	6 each
5	Q	5
6	L	4
7–10	S, A, Z, K	3 each
11–13	C, D, J	2 each
14–20	X, I, U, B, M, T, E	1 each

Table 1: Letter frequencies in the assigned ciphertext.

The three most frequent ranks are **P** (7), then the tie **R, O, F** (6). In English, E, T, A, O, I, N, S, H, R are usually the most common letters, so P is a reasonable first guess for E or S.

2.3 Short English words

Two-letter English words that appear often include **OF**, **TO**, **IN**, **IS**, **IT**, **AS**, and **AN**. Common three-letter words include **THE**, **AND**, **FOR**, and **ARE**.

In the ciphertext, **FP** appears twice. **QDFP FP** is a four-letter word followed by that same pair, which matches the English pattern **THIS IS**. That single guess gives $Q \rightarrow T$, $D \rightarrow H$, $F \rightarrow I$, $P \rightarrow S$, so **FP** really is **IS**, and **QDR** becomes **TH?**, almost certainly **THE** ($R \rightarrow E$).

Those mappings already show that this is *not* a Caesar cipher: **P** maps to **S** (shift 11) while **R** maps to **E** (shift 13). It is a general monoalphabetic substitution.

2.4 Recovering the plaintext

Continuing left to right with a one-to-one table:

- **PRCSOFQX** $\rightarrow$ **SE????ITY** $\rightarrow$ **SECURITY**
- **AFOPQ** $\rightarrow$ **?IRST** $\rightarrow$ **FIRST**
- **CZSPR** $\rightarrow$ **C?USE** $\rightarrow$ **CAUSE**
- **LA** $\rightarrow$ **OF**
- **JFPALOQSKR** $\rightarrow$ **?ISFORTU?E**
 $\rightarrow$ **MISFORTUNE**
- second sentence: **ZK=AN**, **LIU=OLD**, **BROJZK=GERMAN**, **MOLTROE=PROVERB**

Key Takeaway

Plaintext: **SECURITY IS THE FIRST CAUSE OF MISFORTUNE.**
THIS IS AN OLD GERMAN PROVERB.

Cipher letters	P R C S O F Q X D A Z L J K I U B M T E
Plain letters	S E C U R I T Y H F A O M N L D G P V B

Table 2: Recovered substitution. Unused alphabet letters are omitted.

Part 1(c): manual cracking time

Fill in the wall-clock time you personally spent on the pencil-and-paper decrypt *before* submitting, for example “27 minutes”. Do not use the Python runtime. Recorded time: **[YOUR TIME HERE]**.

2.5 Caesar brute-force program

Caesar has only 26 keys, so brute force is cheap. The program `code/caesar_bruteforce.py` does the following:

1. For every shift $0, \dots, 25$, decode each letter as $(c - \text{shift}) \bmod 26$.
2. Score the candidate with English letter-frequency chi-squared plus hits against a small word list.
3. Print all 26 candidates, best first.

A true Caesar sample (the recovered proverb, shifted by 3) is recovered at shift 3:

Best Caesar candidate on a known-shift sample

```
1 ciphertext:
2 VHFXULWB LV WKH ILUVW FDXVH RI PLVIRUWXQH.
3
4 1. shift= 3  chi2= 26.17  words= 7  <-- best
5     SECURITY IS THE FIRST CAUSE OF MISFORTUNE.
```

The same program on the *homework* ciphertext never produces English. The “best” shift still reads as gibberish (SUFVRITA IS TGU ...). That is expected: the assigned text is a mixed substitution, not a constant shift.

How to run it

```
python3 code/caesar_bruteforce.py 'YOUR CIPHERTEXT HERE'
```

3 Part II: Kasiski examination

Vigenère is a polyalphabetic substitution: a repeating keyword chooses a different Caesar shift for each position. Simple frequency counts on the whole ciphertext fail, because the same plaintext letter can encrypt to several ciphertext letters.

Kasiski examination recovers the keyword length without guessing the keyword itself [1].

1. Scan the ciphertext for repeated strings of length at least three.
2. Record the distances (in letters) between those repeats. Repeats usually occur when the same plaintext chunk lines up with the same keyword letters again, so the distance is a multiple of the keyword length d .

3. Take the greatest common divisor of those distances. That GCD is a strong candidate for d (or a small multiple of d).
4. Split the ciphertext into d independent Caesar streams: stream i is every d -th letter starting at i .
5. Attack each stream with ordinary frequency analysis, then reassemble.

Key Takeaway

Kasiski turns Vigenère into “find d , then crack d Caesar ciphers.” The hard polyalphabetic problem collapses once the period is known.

4 Part III: Block-cipher modes on a bitmap

Electronic Code Book (ECB) encrypts each block with the same key and no chaining. NIST still documents ECB, but warns that identical plaintext blocks produce identical ciphertext blocks [4]. This section makes that warning visible.

4.1 Image conversion

Source file: `docs/images/source.jpg`. ImageMagick resized it to exactly 2000×2000 and wrote a 1-bit binary PBM (Netpbm P4) [2]:

Convert to a 2000x2000 binary PBM

```
1 magick docs/images/source.jpg \  
2   -resize 2000x2000! -colorspace Gray -threshold 50% \  
3   pbm:org.pbm
```

The file header is two ASCII lines, then 500,000 bytes of pixels (2000×2000 bits, packed 8 per byte). That body length is a multiple of the AES block size (16 bytes), so `-nopad` is valid with no extra bytes.

4.2 Strip the header, encrypt, restore the header

Encrypting the header would destroy the PBM magic number, so only the pixel body is encrypted [3]. Demonstration key and CBC IV (hex) are stored in `code/run_experiments.py`; they are lab constants, not production secrets.

AES-256-ECB and AES-256-CBC on the pixel body

```
1 openssl enc -aes-256-ecb -in org.x -out enc-ecb.x \  
2   -nosalt -nopad -K 001122...ccddeeff \  
3 openssl enc -aes-256-cbc -in org.x -out enc-cbc.x \  
4   -nosalt -nopad -K 001122...ccddeeff \  
5   -iv 0102030405060708090a0b0c0d0e0f10
```

The original two-line header P4 / 2000 2000 is prepended again so the results open as images.

4.3 Results

Figure 1 is left-to-right: original 1-bit image, AES-256-ECB, and AES-256-CBC.

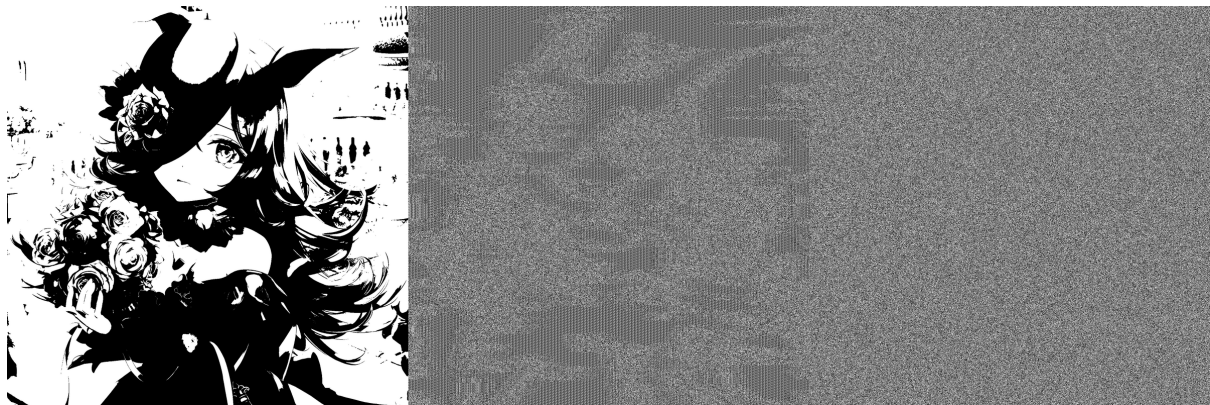


Figure 1: Original 1-bit image (left), AES-256-ECB (middle), AES-256-CBC (right).

Larger individual frames are in docs/images/: `original.png`, `aes256-ecb.png`, and `aes256-cbc.png`.

4.4 What the pictures show

In ECB, repeated 16-byte plaintext regions (flat sky, dark hair, the dress) encrypt to the same ciphertext bytes. After the header is restored, those repeats still draw the outline of the figure. The middle panel is noisy, but the silhouette is still there.

CBC XORs each plaintext block with the previous ciphertext block (the first block uses the IV). Equal plaintext blocks therefore become unequal ciphertext blocks. The right panel is unstructured static: no usable picture remains.

Key Takeaway

A block cipher is only as hiding as its *mode*. ECB leaks repeated structure. Modes with an IV and chaining or feedback (CBC, CTR, GCM, ...) should be used for real data, especially images and other patterned files.

5 Part IV: Hashing, ciphers, and digital signatures

5.1 Experimental design

Performance was measured with OpenSSL's own timing tool [8], not a stopwatch.

- Implementation: OpenSSL 3.6.4 (/opt/homebrew/opt/openssl@3/bin/openssl).
- Host: Apple M1 Pro, otherwise idle.
- Each algorithm ran for 3 seconds per buffer size, three independent times. The table reports the **median** of those three runs.
- SHA-1 used the default provider. RC4 and Blowfish are legacy in OpenSSL 3, so those two runs also loaded `-provider legacy`.
- DSA reports sign and verify rates at 1024-bit and 2048-bit parameters (OpenSSL 3 no longer times 512-bit DSA).

Commands used for Part 4(a)

```
1 openssl speed -seconds 3 sha1
2 openssl speed -seconds 3 -provider legacy -provider default rc4
3 openssl speed -seconds 3 -provider legacy -provider default blowfish
4 openssl speed -seconds 3 dsa
```

OpenSSL prints symmetric throughput in thousands of bytes per second. Table values keep that unit.

5.2 Measured results

On large buffers, SHA-1 hashed about 2.3 GB/s, RC4 encrypted about 1.3 GB/s, and Blowfish stayed near 126 MB/s. DSA is in a different league: a few thousand signatures per second, not gigabytes of bulk data.

Algorithm	16 B	64 B	256 B	1024 B	8192 B	16384 B
SHA-1	111625	391697	1092786	1830493	2316151	2333136
RC4	875828	1140693	1233343	1244972	1261681	1259033
Blowfish	121621	124320	125773	125994	126441	125495

Table 3: Median throughput in 1000s of bytes per second (OpenSSL units).

DSA size	sign (s)	verify (s)	sign/s	verify/s
1024 bit	0.000068	0.000053	14708	18846
2048 bit	0.000193	0.000173	5172	5770

Table 4: Median DSA sign and verify rates.

5.3 Performance versus security

Primitive	Role	Speed here	Security status
SHA-1	Hash	Fastest bulk primitive measured	Collision attacks are practical [7]; NIST has moved the ecosystem off SHA-1 [5]. Use SHA-256 or SHA-3 for new work.
RC4	Stream cipher	Very fast stream encryption	Biased keystream; forbidden in TLS [6].
Blowfish	64-bit block cipher	Much slower than RC4/SHA-1 on this CPU	64-bit blocks are small by modern standards; AES is the usual replacement.
DSA	Public-key signature	Thousands of ops/s, not GB/s	Provides authentication and non-repudiation, not bulk confidentiality. 1024-bit parameters are no longer considered adequate.

Table 5: What each measured primitive is for, and whether it is still appropriate.

Fast is not the same as safe. RC4 won the throughput contest among the ciphers and still must not be used. DSA lost the throughput contest and is still the only primitive in this set that can bind a message to a private key.

5.4 Digital signatures

A digital signature does not encrypt the whole file with DSA. It combines a fast hash with a slow public-key operation:

1. The signer hashes the message (SHA family). The digest is short even if the file is

large.

2. The signer applies the private key to that digest (DSA, RSA, or ECDSA).
3. The verifier hashes the received message independently and checks the signature with the public key.

What each piece contributes

Hash: cheap integrity check; any bit flip changes the digest. **Public-key sign:** only the private-key holder can produce a valid signature (authentication and non-repudiation). **Symmetric cipher (AES, historically RC4/Blowfish):** optional confidentiality for the payload, usually under a random session key.

The combination avoids each tool's weakness. DSA is too slow to encrypt megabytes. A hash alone is not a signature, because anyone can recompute it. A symmetric cipher keeps the bytes secret but does not prove who sent them. Real protocols therefore *hash, sign the digest, and encrypt the body separately* (hybrid encryption plus signatures).

For new designs, replace the lab primitives with SHA-256 (or SHA-3), AES-GCM or ChaCha20-Poly1305, and ECDSA/Ed25519 or RSA-PSS.

References

- [1] Wikipedia contributors. *Kasiski examination*. https://en.wikipedia.org/wiki/Kasiski_examination.
- [2] Wikipedia contributors. *Netpbm*. <https://en.wikipedia.org/wiki/Netpbm>.
- [3] OpenSSL Wiki. *Enc*. <https://wiki.openssl.org/index.php/Enc>.
- [4] National Institute of Standards and Technology. *NIST SP 800-38A: Recommendation for Block Cipher Modes of Operation*, 2001. <https://csrc.nist.gov/publications/detail/sp/800-38a/final>.
- [5] National Institute of Standards and Technology. *NIST Policy on Hash Functions*. <https://csrc.nist.gov/projects/hash-functions/nist-policy-on-hash-functions>.
- [6] A. Popov. *Prohibiting RC4 Cipher Suites*. RFC 7465, 2015. <https://www.rfc-editor.org/rfc/rfc7465>.
- [7] Google and CWI Amsterdam. *SHAttered*. <https://shattered.io/>.
- [8] OpenSSL. *openssl-speed*. <https://docs.openssl.org/3.6/man1/openssl-speed/>.